Algorithm Analysis - US13

Introduction

This document analyzes the **time complexity** of the algorithms implemented in **US13**, which involve the methods `checkReachability` and `transitiveClosure`. These methods deal with determining the reachability in graphs represented by adjacency matrices, where the first method checks if there is a path between two nodes, and the second computes the **transitive closure** of a graph.

The analysis will focus on evaluating the **worst-case time complexity** using Big O notation. This will help us understand the behavior of the algorithms as the graph size (number of nodes and edges) increases, which is critical when applying these algorithms to larger datasets.

---

1. `checkReachability` Method

The `checkReachability` method is designed to determine whether there is a path between two stations, `a` and `b`, in a graph. It does this by utilizing the **transitive closure**, which captures all possible paths in the graph. After calculating the transitive closure, the method simply checks if there is a path between the given nodes.

`checkReachability` Pseudocode Analysis

Operation	Approx. Elementary Operations	Big-O Complexity
`boolean[][] F := transitiveClosure(M)`		**$O(n^3)$**
`if a < 0 or a >= n or b < 0 or b >= n` then	1C	**O(1)**
`write "invalid index"` `return;`	≤1 R	**O(1)**
`else`	≤1	**O(1)**
`if F[a,b] := 1` then	1C	**O(1)**
`write "Path exists between a and b"` `return;`	≤1C	**O(1)**

`else`
`write "No Path between a and b"`
`return;`
	1R	**O(1)**

Total		**$O(\max(O(n^3)+O(1)+...) = O(n^3)$**

Explanation:

1. **Transitive Closure Calculation (`transitiveClosure`)**: The primary operation is the calculation of the transitive closure `F`, which is computed through the `transitiveClosure` method. This step is **$O(n^3)$** due to the triple nested loop that propagates connectivity between all pairs of vertices in the graph.

2. **Reachability Check**: After obtaining the transitive closure, the method simply checks if there is a path between `a` and `b`. This lookup operation is **O(1)**, as it is a direct access to the matrix `F`.

Total Complexity:

The overall time complexity of the `checkReachability` method is **$O(n^3)$**, due to the time required to compute the transitive closure.

---

2. `transitiveClosure` Method

The `transitiveClosure` method computes the transitive closure of a graph, which allows us to determine whether there is a path between any pair of vertices, either directly or through intermediate vertices. This method is the core of the reachability logic in the graph.

`transitiveClosure` Pseudocode Analysis

Operation	Approx. Elementary Operations	Big-O Complexity
`boolean[][] F := M`	1A	**O(1)**
for `k := 0` to `n`	(n+1)A + (n+1)C	**O(n)**
--- **for** `i := 1` to `n`	$(n+1)^2$A + $(n+1)^2$C	**$O(n^2)$**
---- `if F[i,k] := 1` then	n^2C	**$O(n^2)$**
------- **for** `j := 1` to `n`	$(n+1)^3$A + $(n+1)^3$C	**$O(n^3)$**
--------- `F[i][j] := F[i][j] or F[k][j]`	n^3A	**$O(n^3)$**
`return F`	1R	**O(1)**
Total		**$O(\max(O(n^3)+O(n^2)+...) = O(n^3)$**

Explanation:

1. **Initialization**: The method begins by initializing the matrix `F` based on the adjacency matrix `M`. This initialization step takes **$O(n^2)$** time because it involves copying the entire adjacency matrix into the reachability matrix `F`.

2. **Propagation of Connectivity**: The core part of the algorithm consists of three nested loops:

    - The outermost loop (over `k`) represents the intermediate vertex being checked.
    - The second loop (over `i`) checks the reachability from each vertex `i`.
    - The innermost loop (over `j`) checks if the reachability from `i` to `j` is possible through the intermediate vertex `k`.

    The three nested loops, each iterating **n** times, result in **$O(n^3)$** complexity.

Total Complexity:

The time complexity of the `transitiveClosure` method is **O(n³)**, as the nested loops propagate reachability through all pairs of vertices in the graph.

Analysis of Results

Both the **`checkReachability`** and **`transitiveClosure`** methods have a time complexity of **O(n³)**. The reason for this is that both methods rely on the computation of the transitive closure, which involves a triple-nested loop that iterates over all pairs of vertices.

Performance Implications:

1. **Scalability**: As the graph grows larger, with an increasing number of vertices (`n`), the execution time of these algorithms will grow cubically. For large-scale graphs, such as transportation networks with thousands of stations, these algorithms may become inefficient.

2. **Practical Applications**: These algorithms are effective for small to medium-sized graphs. However, for large transportation networks, more efficient algorithms like **Depth-First Search (DFS)** may be preferable, as they have linear time complexity in relation to the number of vertices and edges (O(n + m)).

Conclusion

The time complexity analysis reveals that both the **`checkReachability`** and **`transitiveClosure`** methods have a cubic time complexity of **O(n³)**. This means that for larger graphs, the performance of these algorithms will degrade quickly, which could be a limitation when dealing with large datasets such as transportation networks. In such cases, optimizations or alternative algorithms might be needed to achieve better performance and scalability.